
Particle Flow Filter and Differentiable Particle Filter

Zhongchangfei LI
zlihc@connect.ust.hk

Abstract

This report summarizes classic algorithms for filtering task in state-space model (SSM), where the noisy observations are determined by unobservable hidden states and random observation noise, while the hidden states transition according to some stochastic process. The report covers contents ranging from the most basic Kalman filter for linear Gaussian state-space model (LGSSM) and its classic extensions, to particle filters and its advanced extensions particle flow filters. Differentiable particle filter, which is proposed to enable end-to-end inference of the model parameters, is also discussed and compared with the classic approaches based on expectation-maximization algorithms. The details can be found in the report and the codebase: <https://github.com/FlyingBeyondUp/JPM-MLCOE-2026>.

Contents

1	State-Space Model	2
1.1	General Form	2
1.2	Examples	2
1.2.1	Stochastic Volatility Model	2
1.2.2	Log-Squared Stochastic Volatility Model	3
1.2.3	Vasicek Bond Pricing Model	3
1.3	Summary of the Filters	4
2	Classical Filters	4
2.1	The Kalman Filter	5
2.2	Extended Kalman Filter (EKF)	5
2.3	Unscented Kalman Filter (UKF)	5
2.4	Standard Particle Filter (Bootstrap)	6
3	Particle Flow-Based Methods	7
3.1	A Unified Description via Stochastic Flow	8
3.2	Exact Daum-Huang (EDH) and Local EDH Flows	10
3.3	Invertible Particle Flow Particle Filter (PF-PF)	12
3.4	Kernel Particle Flow Filter	13

4	Learning Model Parameters from Data	17
4.1	Classic Filters	17
4.1.1	Rauch-Tung-Striebel (RTS) Smoother	17
4.1.2	EM Algorithm for LGSSM	17
4.2	Differentiable Particle Filter	17
4.2.1	Entropy-Regularized Optimal Transport	18
4.2.2	Sinkhorn Algorithm	18
4.2.3	Differentiability	19

1 State-Space Model

1.1 General Form

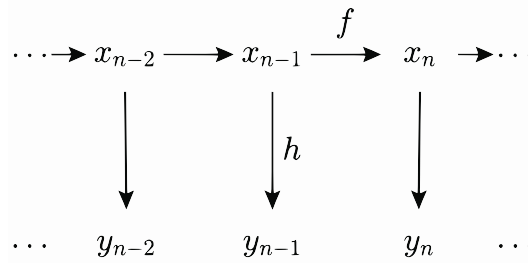


Figure 1: Schematic Figure for State-Space Model

Fig. 1 is a schematic figure of SSM, where the unobservable hidden state x_n at time step n transition according to the dynamics defined by function f and the observations y_n is given by the hidden states according to function h . Denote the transition noise by q_n , and observation noise by r_n , the SSM can be depicted by

$$\begin{aligned} x_n &= f(x_{n-1}, q_{n-1}) \\ y_n &= h(x_{n-1}, r_{n-1}) \end{aligned} \quad (1)$$

Therefore, the probability density $p(x_n|x_{n-1})$ is also called transition density while $p(y_n|x_n)$ is called emission density. The filter task can be modeled as a Bayesian inference problem, if at time step n , we have an approximate posterior probability $p(x_n|y_n)$, then the hidden state can be estimated by $\hat{x}_n \approx E[x_n|y_n]$. The hidden state at the next step can be predicted by the probability distribution

$$p(x_{n+1}|y_n, \dots, y_1) = \int_{\Omega_x} p(x_{n+1}|x_n)p(x_n|y_n, \dots, y_1)dx_n, \quad (2)$$

where Ω_x is the whole space of the hidden state. This prediction can be updated once we observe the y_{n+1}

$$p(x_{n+1}|y_{n+1}, y_n, \dots, y_1) = \frac{p(y_{n+1}|x_{n+1})p(x_{n+1}|y_n, \dots, y_1)}{p(y_{n+1}|y_n, \dots, y_1)}, \quad (3)$$

where $p(x_{n+1}|y_n, \dots, y_1)$ plays the role of prior distribution and the emission density $p(y_{n+1}|x_{n+1})$ is the likelihood that update our estimate of the hidden state at time step $n + 1$. In the remaining of this section, we explain the one-dimensional SSM used in the report to visualize the SSM and the filtered results.

1.2 Examples

1.2.1 Stochastic Volatility Model

The Stochastic Volatility Model is widely used in financial econometrics to model time-varying variance in asset returns. In the codebase, this is implemented via the function `get1DStochasticVolModel`.

Let x_t represent the latent log-volatility and y_t represent the observed asset return (or mean-adjusted return).

The log-volatility follows an autoregressive AR(1) process:

$$x_{t+1} = \alpha x_t + \eta_t, \quad \eta_t \sim \mathcal{D}(0, \sigma^2) \quad (4)$$

where x_t is the latent state (log-volatility), α is the persistence parameter ($|\alpha| < 1$ for stationarity), σ is the volatility of volatility (vol-of-vol) and $\mathcal{D}$ is the process noise distribution. The code supports both Gaussian $\mathcal{N}(0, \sigma^2)$ and heavy-tailed Student-t distributions.

The observations depend non-linearly on the state via an exponential function:

$$y_t = \beta \exp\left(\frac{x_t}{2}\right) \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, 1) \quad (5)$$

where β is the scaling factor for the observation noise (related to baseline volatility) and ϵ_t is the observation noise, which is assumed to be standard normal. The term $\exp(x_t/2)$ represents the instantaneous standard deviation derived from the log-variance x_t .

1.2.2 Log-Squared Stochastic Volatility Model

To utilize linear filtering techniques (like the Kalman Filter) or to simplify the observation density, the SVM can be transformed by squaring and taking the logarithm of the observations. This is implemented in `get1DLogSquaredSVM` and approximated in `get_LogSVM_LGSSM`.

Starting from the observation equation $y_t = \beta \exp(x_t/2) \epsilon_t$, we square both sides:

$$y_t^2 = \beta^2 \exp(x_t) \epsilon_t^2 \quad (6)$$

Taking the logarithm yields a linear observation equation with non-Gaussian noise:

$$\log(y_t^2) = x_t + \log(\beta^2) + \log(\epsilon_t^2) \quad (7)$$

where $\log(\epsilon_t^2)$ follows a $\log\text{-}\chi_1^2$ distribution (logarithm of a Chi-square variable with 1 degree of freedom), with Mean: $\mathbb{E}[\log(\epsilon_t^2)] = \psi(0.5) - \log(0.5) \approx -1.27$, where ψ is the digamma function and $\text{var}[\log(\epsilon_t^2)] = \frac{\pi^2}{2} \approx 4.93$. In the codebase, the $\log\text{-}\chi_1^2$ distribution is approximated by a normal distribution with the same mean and variance for the filtering algorithms explicitly requiring an additive Gaussian noise in the model.

The code constructs an LGSSM approximation by treating the non-Gaussian noise as Gaussian with the moments derived above.

$$x_{t+1} = \alpha x_t + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \sigma^2) \quad (8)$$

$$z_t = x_t + \mu_{\text{bias}} + v_t, \quad v_t \sim \mathcal{N}(0, \frac{\pi^2}{2}) \quad (9)$$

where $z_t = \log(y_t^2)$ and the bias term is $\mu_{\text{bias}} = \log(\beta^2) + \mathbb{E}[\log(\epsilon_t^2)]$. This allows the use of a standard Kalman Filter as a robust baseline.

1.2.3 Vasicek Bond Pricing Model

This model combines the Vasicek short-rate process with the affine term structure of interest rates to price zero-coupon bonds. It is implemented in `getVasicekBondPriceModel`.

The instantaneous short rate r_t follows a mean-reverting Ornstein-Uhlenbeck SDE:

$$dr_t = \kappa(\theta - r_t)dt + \sigma dW_t \quad (10)$$

where κ is the mean reversion rate, θ is the long-term mean level of the interest rate, and σ is the volatility of the short rate. To implement this model numerically, the code defines the latent state as the centered rate $x_t = r_t - \theta$. The exact discretization over time step Δt is:

$$x_{t+1} = e^{-\kappa\Delta t} x_t + \eta_t, \quad \eta_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\kappa}(1 - e^{-2\kappa\Delta t})\right) \quad (11)$$

For small Δt , the code approximates the variance as $\sigma^2 \Delta t$.

The observation y_t is the price of a Zero-Coupon Bond with time to maturity τ . Under the Vasicek model, this price is given by the affine formula:

$$P(r_t, \tau) = A(\tau)e^{-B(\tau)r_t} \quad (12)$$

Substituting $r_t = x_t + \theta$:

$$y_t = P(x_t + \theta, \tau) + \nu_t = A(\tau)e^{-B(\tau)(x_t + \theta)} + \nu_t \quad (13)$$

The coefficients are:

$$B(\tau) = \frac{1 - e^{-\kappa\tau}}{\kappa} \quad (14)$$

$$A(\tau) = \exp\left(\left(\theta - \frac{\sigma^2}{2\kappa^2}\right)(B(\tau) - \tau) - \frac{\sigma^2}{4\kappa}B(\tau)^2\right) \quad (15)$$

This constitutes a non-linear observation function $h(x_t)$ suited for EKF/UKF testing.

1.3 Summary of the Filters

The classic Kalman filter requires linear model and additive Gaussian noise, while the extend Kalman filter (EKF) and unscented Kalman filter (UKF) generalize the classic one to deal with nonlinear models by local linearization (EKF) or statistic moment matching (UKF), which still fail for highly nonlinear tasks. Particle Filter do not make assumptions on the model but use a large number of particles to approximate the model distributions, which can be applicable to nonlinear model but will fail when the task is a high dimensional problem, in which case a large number of particles will be placed in the region with only tiny probability distribution. Resampling can solve the particle degeneracy problem but will lead to sample impoverishment problem, where the particle collapse with a small effective sample size. Thus, particle flow filters are proposed to avoid using explicit weights and resampling, but approach the posterior distribution by dynamics.

2 Classical Filters

Classical filters enable analytical solutions for state estimation but require linear transition and observation models, as well as Gaussian process and observation noise. We define a time-invariant Linear Gaussian State Space Model (LGSSM) as

$$\begin{aligned} x_t &= Ax_{t-1} + q_t, & q_t &\sim \mathcal{N}(0, Q) \\ y_t &= Cx_t + r_t, & r_t &\sim \mathcal{N}(0, R) \end{aligned} \quad (16)$$

where $x_t \in \mathbb{R}^{D_x}$ is the latent state, $y_t \in \mathbb{R}^{D_y}$ is the observation, A is the state transition matrix, and C is the observation matrix. The initial state is distributed as $x_0 \sim \mathcal{N}(\mu_0, P_0)$.

The schematic figures for filter and smoother can be found in Fig. 2.

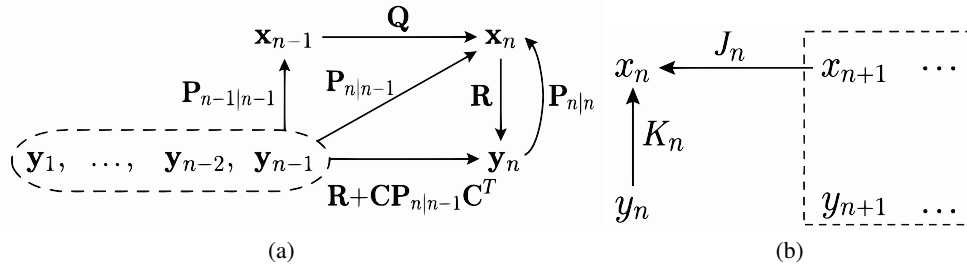


Figure 2: Schematic Figure for the relations between covariance matrices (a) and information flow in filter and smoother (b).

At each time step n , given the estimated state $\mathbf{x}_{n-1}$ and the past observations $y_{1:n-1}$, the filter predicts the current state $\mathbf{x}_n$ and covariance $P_{n|n-1}$ first, and then updates the estimates based on the

new observation y_n , as shown in Fig. 2(a). On the other hand, as shown in Fig. 2(b), if the whole sequence of observations have been observed, the estimated states can be further updated and the resultant sequence of states would be smoothed. In this section, we focus on the mechanism of filters, while the smoother will be discussed in detail in Section. 4.

2.1 The Kalman Filter

The Kalman Filter performs exact Bayesian inference to estimate the posterior distribution $p(x_t|y_{1:t})$. In the codebase, this is implemented within the `KalmanFilter` class. Given the posterior at time $t-1$, denoted by $x_{t-1|t-1} \sim \mathcal{N}(\hat{x}_{t-1|t-1}, P_{t-1|t-1})$, the predictive distribution is derived by taking expectations of the state transition equation

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1} \quad (17)$$

$$P_{t|t-1} = AP_{t-1|t-1}A^\top + Q \quad (18)$$

Upon observing y_t , the estimate is corrected by computing the innovation $\nu_t = y_t - C\hat{x}_{t|t-1}$ and the innovation covariance $S_t = CP_{t|t-1}C^\top + R$. The optimal Kalman Gain K_t , derived by minimizing the mean squared error, is given by $K_t = P_{t|t-1}C^\top S_t^{-1}$. Consequently, the state estimate and covariance are updated as

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t\nu_t \quad (19)$$

The standard covariance update $P_{t|t} = (I - K_tC)P_{t|t-1}$ is prone to numerical errors, potentially resulting in non-symmetric or indefinite matrices. To ensure positive semi-definiteness, the implementation utilizes the Joseph Form, given by $P_{t|t} = (I - K_tC)P_{t|t-1}(I - K_tC)^\top + K_tRK_t^\top$, whose effect is shown in Fig. 10, where the Kalman filter is tested for a linear Gaussian SSM with ten state and observation dimensions. It can be found in the `checkKF.py` file in the codebase. This formulation arises from expanding the error covariance definition $\mathbb{E}[(x_t - \hat{x}_{t|t})(x_t - \hat{x}_{t|t})^\top]$.

2.2 Extended Kalman Filter (EKF)

For non-linear systems, the linearity assumption of the Kalman Filter does not hold. The EKF addresses this by linearizing the dynamics around the current estimate. This is implemented in the `ExtendedKalmanFilter` class. The system is described by non-linear transition and observation functions

$$\begin{aligned} x_t &= f(x_{t-1}, q_t) \\ y_t &= h(x_t, r_t) \end{aligned} \quad (20)$$

The EKF approximates these functions via a first-order Taylor expansion. The implementation utilizes Automatic Differentiation to compute the Jacobians $F_t = \frac{\partial f}{\partial x} \big|_{\hat{x}_{t-1|t-1}}$ and $H_t = \frac{\partial h}{\partial x} \big|_{\hat{x}_{t|t-1}}$. The standard Kalman filter prediction and update equations are then applied, substituting F_t for A and H_t for C .

Since the exact posterior is intractable in non-linear models, the implementation uses an approximate EM approach. In the E-Step, we run the EKF Smoother (linearized RTS) to obtain approximations for $\hat{x}_{t|T}$ and $P_{t|T}$. In the M-Step, we update Q and R based on the residuals of the non-linear functions. For example, the observation noise covariance is updated as

$$R_{new} \approx \frac{1}{T} \sum_t ((y_t - h(\hat{x}_{t|T}))(y_t - h(\hat{x}_{t|T}))^\top + H_t P_{t|T} H_t^\top) \quad (21)$$

2.3 Unscented Kalman Filter (UKF)

The EKF can perform poorly when non-linearities are severe. The Unscented Kalman Filter (UKF) captures higher-order moments by propagating a set of deterministic Sigma Points through the non-linear functions. This is implemented in `UnscentedKalmanFilter`. For an n -dimensional state with mean $\bar{x}$ and covariance P , we generate $2n + 1$ sigma points. Defining a scaling parameter $\lambda = \alpha^2(n + \kappa) - n$, the points are given by

$$\begin{aligned} \mathcal{X}_0 &= \bar{x} \\ \mathcal{X}_i &= \bar{x} + \left(\sqrt{(n + \lambda)P} \right)_i, \quad i = 1 \dots n \\ \mathcal{X}_{i+n} &= \bar{x} - \left(\sqrt{(n + \lambda)P} \right)_i, \quad i = 1 \dots n \end{aligned} \quad (22)$$

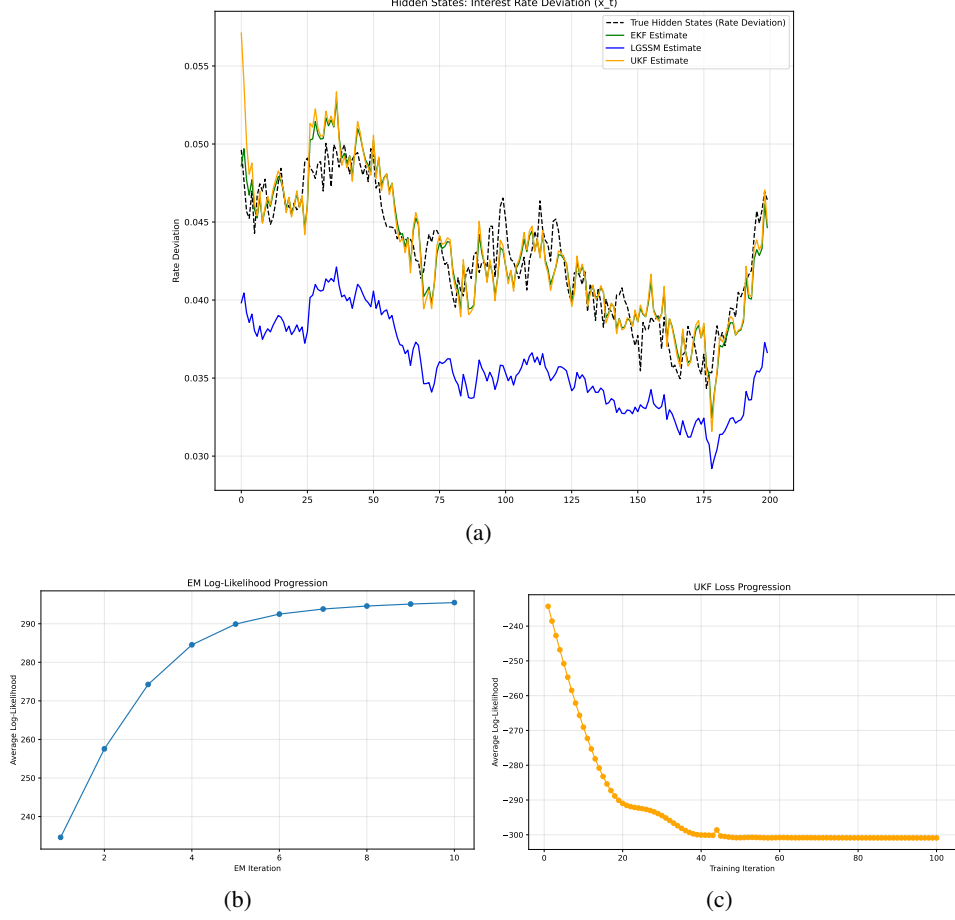


Figure 3: (a) Comparison between Kalman filter, Extend Kalman filter (EKF) and Unscented Kalman filter (UKF) on the Vasicek. (b) and (c) example for the convergence of EM algorithm for EKF and gradient descent for UKF.

The weights for recovering the mean (W_m) and covariance (W_c) are defined such that $W_m^{(0)} = \frac{\lambda}{n+\lambda}$ and $W_c^{(0)} = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta)$, while for $i > 0$, $W_m^{(i)} = W_c^{(i)} = \frac{1}{2(n+\lambda)}$. Here β incorporates prior knowledge of the distribution.

In the prediction step, the sigma points are propagated through the transition function $f(\cdot)$ to yield $\mathcal{Y}_i = f(\mathcal{X}_i)$, and the predicted mean is $\hat{x}_{pred} = \sum W_m^{(i)} \mathcal{Y}_i$. In the update step, these points are mapped to observation space via $h(\cdot)$, yielding $\mathcal{Z}_i = h(\mathcal{Y}_i)$ and $\hat{y}_{pred} = \sum W_m^{(i)} \mathcal{Z}_i$. The Kalman Gain is computed utilizing the cross-covariance P_{xy} and innovation covariance S as $K = P_{xy}S^{-1}$.

Unlike the EM algorithm used for LGSSM, the UKF implementation employs direct gradient descent to optimize parameters, specifically the log-scales of process and observation noise. The objective function is the negative log-likelihood of the observations $\mathcal{L} = -\sum_{t=1}^T \log p(y_t|y_{1:t-1})$, where $p(y_t|y_{1:t-1}) \approx \mathcal{N}(y_t; \hat{y}_{t|t-1}, S_t)$. This approach facilitates learning stationary parameters in highly non-linear settings where analytic M-steps are intractable.

The comparison between KF, EKF and UKF can be found in Fig. 3 and the `nonlinearSSM.py` in the codebase.

2.4 Standard Particle Filter (Bootstrap)

The Bootstrap Particle Filter (BPF) serves as the standard baseline implementation in `particle_filter.py`. It utilizes the system transition density as the proposal distribution, given

by $q(x_k|x_{k-1}, y_k) = p(x_k|x_{k-1})$. Given a set of particles $\{x_{k-1}^i, w_{k-1}^i\}_{i=1}^N$ approximating $p(x_{k-1}|y_{1:k-1})$, the algorithm proceeds by first propagating samples $x_k^i \sim p(x_k|x_{k-1}^i)$. Subsequently, the unnormalized importance weights are updated via the likelihood function according to

$$\tilde{w}_k^i \propto w_{k-1}^i \frac{p(y_k|x_k^i)p(x_k^i|x_{k-1}^i)}{q(x_k^i|x_{k-1}^i, y_k)} = w_{k-1}^i p(y_k|x_k^i) \quad (23)$$

To mitigate degeneracy, the particles are resampled using Multinomial or Systematic resampling when the Effective Sample Size (ESS) drops below a defined threshold. The example for particle filter and its weight degeneracy problem for tasks with high measurement precision can be found in Fig. 4 and Fig. 5.

Here the effective sample size is calculated by $ESS = \frac{1}{\sum_i (\omega^i)^2}$.

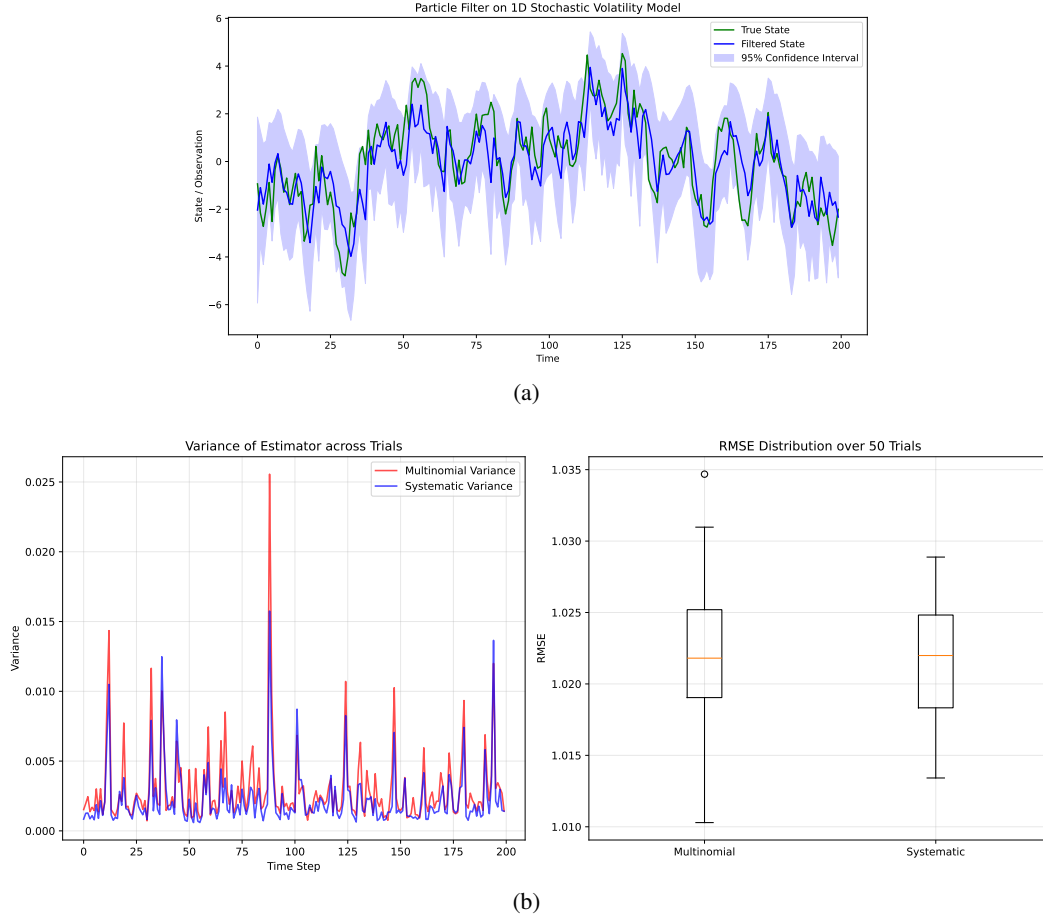


Figure 4: (a) Particle filter for 1D stochastic volatility model. (b) Comparison between multinomial resampling and systematic resampling.

3 Particle Flow-Based Methods

Nonlinear filtering aims to estimate the posterior distribution $p(x_k|y_{1:k})$ of a state x_k given observations $y_{1:k}$. Standard sequential Monte Carlo (SMC) methods often suffer from weight degeneracy in high-dimensional spaces or when measurements are highly informative. In such cases, the proposal distribution, which is often chosen to be the prior, has little overlap with the likelihood, causing most particle weights to collapse to zero. To address this, the implemented algorithms utilize Particle Flow, which migrates particles from the prior to the posterior via a pseudo-time process before the weight update step.

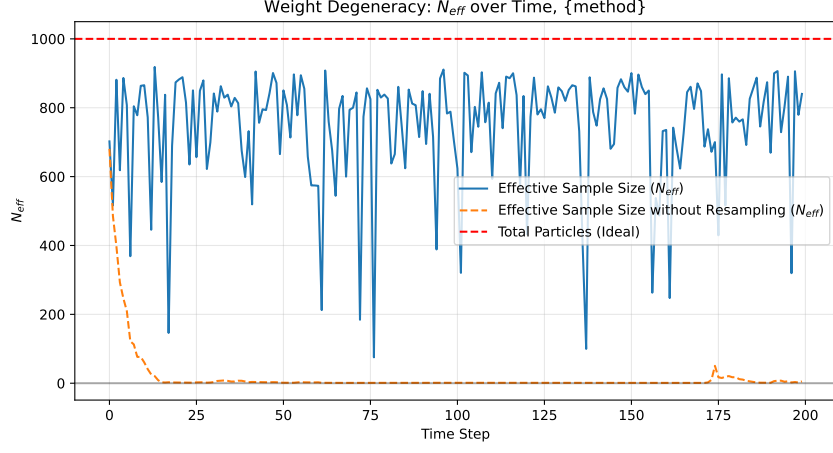


Figure 5: Weight degeneracy problem of particle filter without resampling

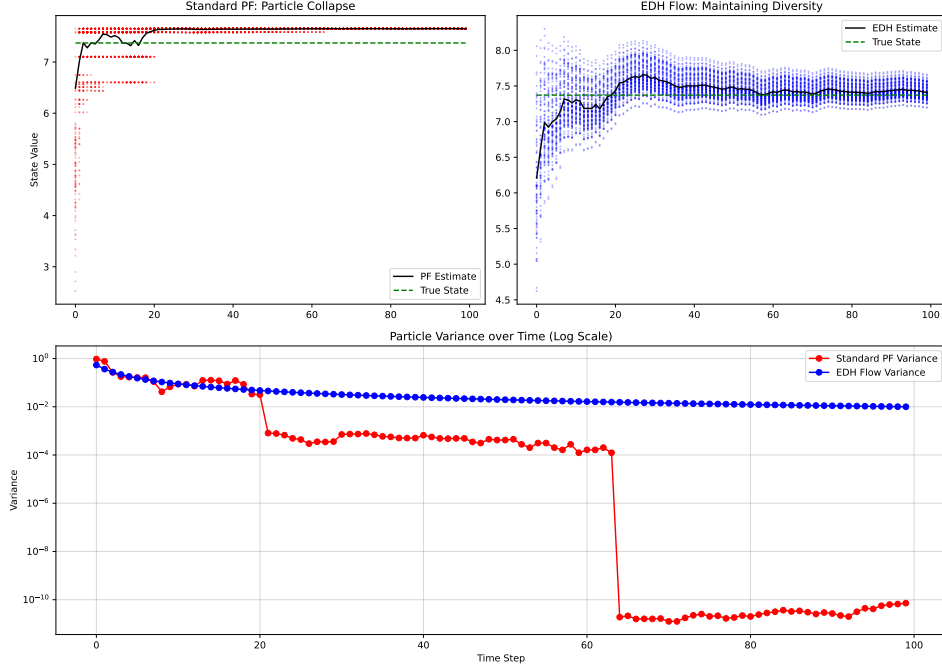


Figure 6: Sample Impoverishment problem for particle filter with resampling

3.1 A Unified Description via Stochastic Flow

This section aims to: 1. Unify particle flows as special cases of a parameterized stochastic PFF. 2. Show that for linear measurement with Gaussian noise, the PDFs are unbiased. 3. Establish finite time stability (Lyapunov-like).

The parameter is resulted from introducing a nonzero diffusion process to drive the flows as a scheme to improve numerical stability in implementation. For state x and measurement z :

$$p(x|z) = \frac{p(z|x)p(x)}{p(z)}, \quad p(z) = \int p(z|x)p(x)dx \quad (24)$$

Focusing on one-step Bayesian estimation, denote:

$$\begin{aligned} p_1(x) &= p(x|z) \quad (\text{target posterior}) \\ p_0(x) &= p(x), \quad h(x) = p(z|x) \end{aligned}$$

Particle flows are defined through homotopy:

$$p(x, \lambda) = \frac{p_0^{\alpha(\lambda)}(x)p_1^{\beta(\lambda)}(x)}{Z(\lambda)} \quad (25)$$

where $Z(\lambda)$ is a normalization factor, and $p(x, 1) = p_1(x)$. Thus, $p(x, \lambda)$ defines a homotopy from $p_0(x)$ to $p_1(x)$. Taking the logarithm:

$$\log p(x, \lambda) = \alpha(\lambda) \log p_0(x) + \beta(\lambda) \log p_1(x) - \log Z(\lambda) \quad (26)$$

The stochastic flow $x(\lambda)$ is:

$$dx = f(x, \lambda)d\lambda + q(x, \lambda)dw_\lambda \quad (27)$$

where $f(x, \lambda) \in \mathbb{R}^n$, $q(x, \lambda) \in \mathbb{R}^{n \times m}$ is the diffusion matrix, and $w_\lambda \in \mathbb{R}^m$ is an m -dimensional Brownian motion with $\mathbb{E}[dw_\lambda dw_\lambda^T] = \kappa(\lambda)d\lambda$.

Assume $\kappa(\lambda) = I_{m \times m}$ and denote $Q(x, \lambda) = q(x, \lambda)q(x, \lambda)^T$. Then $\mathbb{E}[q(x, \lambda)dw_\lambda dw_\lambda^T q^T(x, \lambda)] = Q(x, \lambda)d\lambda$. The goal is to select $f(x, \lambda)$ and $q(x, \lambda)$ ($Q(x, \lambda)$) such that:

$$\log p(x, \lambda) = \alpha(\lambda) \log p_0(x) + \beta(\lambda) \log p_1(x) - \log Z(\lambda) \quad (28)$$

is maintained for the particle flow $x(\lambda)$ driven by $dx = f(x, \lambda)d\lambda + q(x, \lambda)dw_\lambda$. Taking the derivative of the target homotopy log density:

$$\frac{\partial \log p}{\partial \lambda} = \dot{\alpha} \log p_0 + \dot{\beta} \log p_1 - \frac{\dot{Z}}{Z} \quad (29)$$

$$= \dot{\alpha} \log p_0 + \dot{\beta}(\log p_0 + \log h + \text{const}) - \frac{\dot{Z}}{Z} \quad (30)$$

$$= (\dot{\alpha} + \dot{\beta}) \log p_0 + \dot{\beta} \log h - \frac{\dot{Z}}{Z} + \text{const} \quad (31)$$

From the Fokker-Planck equation:

$$\frac{\partial p}{\partial \lambda} = -\nabla_x \cdot (fp) + \frac{1}{2} \sum_{i,j} \frac{\partial^2}{\partial x_i \partial x_j} [Q_{ij}p] \quad (32)$$

$$\frac{\partial \log p}{\partial \lambda} = -\frac{1}{p}(f \cdot \nabla_x p + p \nabla_x \cdot f) + \frac{1}{2p} \sum_{i,j} \frac{\partial^2}{\partial x_i \partial x_j} [Q_{ij}p] \quad (33)$$

$$= -f^T \nabla_x \log p - \nabla_x \cdot f + \frac{1}{2p} \sum_{i,j} \frac{\partial^2}{\partial x_i \partial x_j} [Q_{ij}p] \quad (34)$$

Matching the two derivatives gives:

$$(\dot{\alpha} + \dot{\beta}) \nabla_x \log p_0 + \dot{\beta} \nabla_x \log h = -(\nabla_x f)^T (\nabla_x \log p) - [\nabla_x \nabla_x^T \log p] f + \dots \quad (35)$$

Choose $\alpha(\lambda) + \beta(\lambda) = 1$, so $\dot{\alpha} + \dot{\beta} = 0$. We propose an ansatz $f = K_1(\nabla_x \log p) + K_2(\nabla_x \log h)$, and assume $\nabla_x \cdot f = 0$. Matching the $\nabla_x \log h$ terms yields:

$$K_2 = -\dot{\beta}[\nabla_x \nabla_x^T \log p]^{-1} = -\dot{\beta}S^{-1} \quad (36)$$

where $S = \nabla_x \nabla_x^T \log p$, and we let $H = \nabla_x \nabla_x^T \log h$.

Assume Q is independent of x . The diffusion term evaluates to:

$$\nabla_x \left[\frac{1}{2p} \sum_{i,j} \frac{\partial^2}{\partial x_i \partial x_j} (pQ_{ij}) \right] = \nabla_x \left[\frac{1}{2p} \sum_{i,j} Q_{ij} \frac{\partial^2 p}{\partial x_i \partial x_j} \right] \quad (37)$$

$$= \nabla_x \left[\frac{1}{2p} \text{Tr}(Q \nabla_x \nabla_x^T p) \right] \quad (38)$$

Using identities $\nabla_x \log p = \frac{1}{p} \nabla_x p$ and $\nabla_x \nabla_x^T p = p \nabla_x \nabla_x^T \log p + p(\nabla_x \log p)(\nabla_x \log p)^T$:

$$\frac{1}{p} \text{Tr}(Q \nabla_x \nabla_x^T p) = \text{Tr}(Q[(\nabla_x \log p)(\nabla_x \log p)^T + S]) \quad (39)$$

$$= (\nabla_x \log p)^T Q (\nabla_x \log p) + \text{Tr}(QS) \quad (40)$$

Taking the gradient of this expression (using $\nabla_x(u^T Q u) = 2(\nabla_x u)^T Q u$):

$$\nabla_x \left[\frac{1}{2p} \text{Tr}(Q \nabla_x \nabla_x^T p) \right] = (\nabla_x \nabla_x^T \log p) Q (\nabla_x \log p) + \frac{1}{2} \nabla_x \text{Tr}(QS) \quad (41)$$

The entire Right Hand Side (RHS) of the matched equation is:

$$-(\nabla_x f)^T \nabla_x \log p - Sf + SQ \nabla_x \log p + \frac{1}{2} \nabla_x \text{Tr}(QS) = 0 \quad (42)$$

Further assume Q is independent of x , and that $\nabla \log h$ and $\nabla \log p_0$ are linear in x . Since $f = K_1 \nabla_x \log p + K_2 \nabla_x \log h$, its derivative is $\nabla_x f = K_1 S + K_2 H$. Substituting this back:

$$0 = -SK_1 \nabla_x \log p - HK_2 \nabla_x \log p - SK_1 \nabla_x \log p + SQ \nabla_x \log p \quad (43)$$

This leads to $2SK_1 = -HK_2 + SQ$, where $K_2 = -\dot{\beta}S^{-1}$.

$$K_1 = \frac{\dot{\beta}}{2} S^{-1} H S^{-1} + \frac{1}{2} Q \quad (44)$$

For a straight line homotopy $\beta(\lambda) = \lambda$, we have $\dot{\beta} = 1$, making $K_2 = -S^{-1}$. Thus:

$$f = \left(\frac{1}{2} S^{-1} H S^{-1} + \frac{1}{2} Q \right) \nabla_x \log p - S^{-1} \nabla_x \log h \quad (45)$$

If $Q = -S^{-1} H S^{-1}$, then $f = -S^{-1} \nabla_x \log h$.

3.2 Exact Daum-Huang (EDH) and Local EDH Flows

The Exact Daum-Huang (EDH) Particle Flow, implemented in `particle_flow.py`, migrates particles along a pseudo-time $\lambda \in [0, 1]$ according to a stochastic differential equation $d\eta_\lambda = \zeta(\eta_\lambda, \lambda)d\lambda$, where η_0 follows the predictive prior and η_1 approximates the posterior. The drift ζ is derived to satisfy the Fokker-Planck equation such that the distribution of η_λ matches a log-homotopy between the prior and the posterior densities. Assuming a linear-Gaussian approximation, the EDH filter derives an analytic solution for the drift $\zeta(\eta, \lambda) = A(\lambda)\eta + b(\lambda)$. The flow parameters are computed as

$$A(\lambda) = -\frac{1}{2} P H(\lambda)^T (\lambda H(\lambda) P H(\lambda)^T + R)^{-1} H(\lambda) \quad (46)$$

and

$$b(\lambda) = (I + 2\lambda A(\lambda)) [(I + \lambda A(\lambda)) P H(\lambda)^T R^{-1} (y_k - e(\lambda)) + A(\lambda) \bar{\eta}_0] \quad (47)$$

where P is the predictive covariance estimated via EKF or UKF, R is the measurement noise covariance, $H(\lambda)$ is the Jacobian of the measurement function linearized at the mean particle $\bar{\eta}_\lambda$, and $e(\lambda)$ is the linearization error correction. The exact Daum-Huang can be derived as an approximation and deterministic limit of the stochastic flow explained above. Since Q is independent of x , and $\nabla \log p$ and $\nabla \log h$ are linear in x , f is linear in x :

$$f(x, \lambda) = F(\lambda)x + b(\lambda) \quad \text{where} \quad F(\lambda) = \nabla_x f \quad (48)$$

$$F(\lambda) = K_1 S + K_2 H = \left(\frac{1}{2} Q + \frac{\dot{\beta}}{2} S^{-1} H S^{-1} \right) S - \dot{\beta} S^{-1} H \quad (49)$$

$$= \frac{1}{2} Q S - \frac{\dot{\beta}}{2} S^{-1} H \quad (50)$$

The offset vector $b(\lambda)$ is:

$$b(\lambda) = f - F(\lambda)x = \left(\frac{1}{2} Q + \frac{\dot{\beta}}{2} S^{-1} H S^{-1} \right) \nabla_x \log p - \dot{\beta} S^{-1} \nabla_x \log h - \frac{1}{2} Q S x + \frac{\dot{\beta}}{2} S^{-1} H x \quad (51)$$

If $Q = 0$ (deterministic flow) and $\beta = \lambda$, $\log p = \log p_0 + \lambda \log h$. Then $F(\lambda) = -\frac{1}{2}S^{-1}H$, and:

$$b(\lambda) = \frac{1}{2}S^{-1}HS^{-1}\nabla_x \log p - S^{-1}\nabla_x \log h + \frac{1}{2}S^{-1}Hx \quad (52)$$

$$= -F(\lambda)S^{-1}\nabla_x \log p - S^{-1}\nabla_x \log h - F(\lambda)x \quad (53)$$

Assume a Gaussian prior $p(x) = \mathcal{N}(x_g, P_g)$ and linear measurement $z = Hx + v$, with $v \sim \mathcal{N}(0, R)$. The likelihood Hessian is $\nabla_x \nabla_x^T \log h = -H^T R^{-1}H$, and:

$$S = \nabla_x \nabla_x^T \log p = -(P_g^{-1} + \lambda H^T R^{-1}H) \implies S^{-1} = -(P_g^{-1} + \lambda H^T R^{-1}H)^{-1} \quad (54)$$

Rewriting $f = A(\lambda)x + b(\lambda)$, the matrix $A(\lambda)$ is:

$$A(\lambda) = -\frac{1}{2}(P_g^{-1} + \lambda H^T R^{-1}H)^{-1}(H^T R^{-1}H) \quad (55)$$

If the process noise $\rightarrow 0$, $P_g \rightarrow 0$, this equation is numerically unstable, and we have to invert an $n \times n$ matrix $(P_g^{-1} + \lambda H^T R^{-1}H)$, which is too slow in a high-dimensional state space.

We can utilize the Woodbury matrix identity:

$$(P_g^{-1} + \lambda H^T R^{-1}H)^{-1} = P_g - P_g H^T \left(\frac{1}{\lambda} R + H P_g H^T \right)^{-1} H P_g \quad (56)$$

$$= P_g - \lambda P_g H^T (R + \lambda H P_g H^T)^{-1} H P_g \quad (57)$$

This allows us to rewrite $A(\lambda)$ as:

$$A(\lambda) = -\frac{1}{2}P_g H^T (R + \lambda H P_g H^T)^{-1} H \quad (58)$$

This formulation is much more efficient as it only requires the inversion of an $n_{obs} \times n_{obs}$ matrix rather than an $n_{state} \times n_{state}$ matrix.

The gradients are:

$$\nabla_x \log p = -P_g^{-1}(x - x_g) + \lambda H^T R^{-1}(z - Hx) = S(x - \bar{x})$$

$$\nabla_x \log h = H^T R^{-1}(z - Hx)$$

Then $A(\lambda) = -\frac{1}{2}S^{-1}\nabla_x \nabla_x^T \log h$. The vector $b(\lambda)$ becomes:

$$b(\lambda) = f(x, \lambda) - A(\lambda)x = -A(\lambda)S^{-1}\nabla_x \log p - S^{-1}\nabla_x \log h - A(\lambda)x \quad (59)$$

$$= -A(x - \bar{x}) - S^{-1}H^T R^{-1}(z - Hx) - Ax \quad (60)$$

$$= A\bar{x} - S^{-1}H^T R^{-1}z + (S^{-1}H^T R^{-1}H - 2A)x \quad (61)$$

The last term $(S^{-1}H^T R^{-1}H - 2A)x$ is zero because $A = \frac{1}{2}S^{-1}H^T R^{-1}H$. So:

$$b(\lambda) = A\bar{x} - S^{-1}H^T R^{-1}z \quad (62)$$

Using $\bar{x} = \mathbb{E}[x|z] = -S^{-1}(P_g^{-1}x_g + \lambda H^T R^{-1}z)$ w.r.t $p(x, \lambda)$:

$$b(\lambda) = -AS^{-1}(P_g^{-1}x_g + \lambda H^T R^{-1}z) - S^{-1}H^T R^{-1}z \quad (63)$$

$$= -AS^{-1}P_g^{-1}x_g - AS^{-1}\lambda H^T R^{-1}z - S^{-1}H^T R^{-1}z \quad (64)$$

We introduce the identity $-S^{-1} = (P_g^{-1} + \lambda H^T R^{-1}H)^{-1} = (I + 2\lambda A)P_g$. Note that since $A = -\frac{1}{2}P_g H^T (R + \lambda H P_g H^T)^{-1} H$, $(I + 2\lambda A)P_g = P_g + 2\lambda A P_g = -S^{-1}$ by the Woodbury identity expansion above. From this, $-AS^{-1} = (I + 2\lambda A)A P_g$. Substituting these into $b(\lambda)$:

$$b(\lambda) = (I + 2\lambda A)A x_g + (I + 2\lambda A)A P_g \lambda H^T R^{-1}z + (I + 2\lambda A)P_g H^T R^{-1}z \quad (65)$$

$$= (I + 2\lambda A) [(I + \lambda A)P_g H^T R^{-1}z + A x_g] \quad (66)$$

This form does not need to invert the matrix, and R only needs to be calculated once, making it numerically stable even if $P_g \rightarrow 0$.

The standard EDH flow linearizes the measurement model at the global mean, which may be insufficient for highly non-linear functions. The Local Exact Daum-Huang (LEDH) flow addresses this by linearizing the model locally for each particle. Consequently, the drift parameters $A^i(\lambda)$ and $b^i(\lambda)$ become particle-specific, computed using the Jacobian H^i evaluated at each particle's location. While this significantly increases computational complexity, it provides a more accurate flow in non-linear manifolds.

3.3 Invertible Particle Flow Particle Filter (PF-PF)

While EDH and LEDH migrate particles to high-likelihood regions, accumulated discretization errors and Gaussian approximations mean the particles may not exactly sample the true posterior. The PF-PF framework treats the flow as a sophisticated proposal distribution q within a standard Importance Sampling framework. Computing the weights requires evaluating the proposal density $q(\eta_1^i)$, which in turn implies the flow mapping T must be invertible. The flow is discretized into steps of size ϵ_j , creating a composition of affine maps $\eta_{\lambda_j}^i = (I + \epsilon_j A_j^i) \eta_{\lambda_{j-1}}^i + \epsilon_j b_j^i$. For sufficiently small ϵ_j , the matrix $(I + \epsilon_j A_j^i)$ is full rank, ensuring invertibility.

The migrated particle η_1^i after the particle flow process is viewed as being drawn from a proposal distribution $q(\eta_1^i | \chi_{k-1}^i, z_k)$. If the flow process defines an invertible deterministic mapping T , where $\eta_1^i = T(\eta_0^i; z_k, \chi_{k-1}^i)$, then the proposal density is:

$$q(\eta_1^i | \chi_{k-1}^i, z_k) = \frac{p(\eta_0^i | \chi_{k-1}^i)}{|\dot{T}(\eta_0^i; z_k, \chi_{k-1}^i)|} \quad (67)$$

where $\dot{T}$ is the Jacobian of the mapping function $T(\cdot)$.

Since the proposal distribution $q(\eta_1^i | \chi_{k-1}^i, z_k)$ differs from the true distribution $p(\eta_1^i | \chi_{k-1}^i, z_k) = \frac{p(z_k | \eta_1^i) p(\eta_1^i | \chi_{k-1}^i)}{p(z_k)}$, the importance weight of the particle i should be updated as:

$$w_k^i \propto \frac{p(z_k | \eta_1^i) p(\eta_1^i | \chi_{k-1}^i)}{q(\eta_1^i | \chi_{k-1}^i, z_k)} w_{k-1}^i \quad (68)$$

$$= \frac{p(\eta_1^i | \chi_{k-1}^i) p(z_k | \eta_1^i)}{p(\eta_0^i | \chi_{k-1}^i)} |\dot{T}(\eta_0^i; z_k, \chi_{k-1}^i)| w_{k-1}^i \quad (69)$$

To apply the weight updating, the particle flow $T(\cdot)$ must be invertible. The Jacobian determinant is tracked continuously as:

$$|\dot{T}(\eta_0^i; z_k, \chi_{k-1}^i)| = \left| \det \left(\frac{d\eta_1^i}{d\eta_0^i} \right) \right| = \prod_{j=1}^{N_\lambda} |\det(I + \epsilon_j A_j^i(\lambda))| \quad (70)$$

The particle flow procedure for each particle i requires a predicted covariance estimate P^i . At each time step k , for each particle i , the state is projected as $(\chi_{k-1}^i, P_{k-1}^i) \rightarrow (m_{k|k-1}^i, P^i)$ by EKF/UKF. The step-by-step implementation is:

1. Propagate particles with process noise $\eta_0^i = g_k(\chi_{k-1}^i, v_k)$, set $\eta_1^i = \eta_0^i$, and set determinant accumulator $\theta^i = 1$.
2. Set up an auxiliary particle without noise for linearization: $\bar{\eta}_0^i = g_k(\chi_{k-1}^i, 0)$. At $\lambda = 0$, set $\bar{\eta}^i = \bar{\eta}_0^i$.
3. For $\lambda = \lambda + \epsilon_j$:

- Compute the local flow coefficient:

$$A^i(\lambda) = -\frac{1}{2} P^i H^i(\lambda)^T (\lambda H^i(\lambda) P^i H^i(\lambda)^T + R)^{-1} H^i(\lambda)$$

- The linearization is performed at the auxiliary particle $\bar{\eta}^i$, with evaluation error $e^i(\lambda) = h(\bar{\eta}^i, 0) - H^i(\lambda) \bar{\eta}^i$.
- Compute the local drift term:

$$b^i(\lambda) = (I + 2\lambda A^i(\lambda)) [(I + \lambda A^i(\lambda)) P^i H^i(\lambda)^T R^{-1} (z - e^i(\lambda)) + A^i(\lambda) \bar{\eta}_0^i]$$

- Migrate the auxiliary particle: $\bar{\eta}^i = \bar{\eta}^i + \epsilon_j (A_j^i(\lambda) \bar{\eta}^i + b_j^i(\lambda))$.
- Migrate the target particle: $\eta_1^i = \eta_1^i + \epsilon_j (A_j^i(\lambda) \eta_1^i + b_j^i(\lambda))$.
- Accumulate the flow determinant: $\theta^i = \theta^i \cdot |\det(I + \epsilon_j A_j^i(\lambda))|$.

The comparison of the EDH, LEDH and PF-PF, etc. algorithms is shown in the following table. Here, the state-space model is a large spatial sensor networks with d sensors arranged as a $\sqrt{d} \times \sqrt{d}$ grids. Each sensor with index c has a state x_k^c at time k , and the measurement of the sensor is a noisy observation z_k^c . The dynamic model of the system is

$$\begin{aligned} x_k &= \alpha x_{k-1} + v_k \\ z_k &= x_k + w_k \end{aligned} \quad (71)$$

where $\alpha = 0.9$, $v_k = \sigma_z \mathbf{I}_d$ is the measurement noise, and the process noise v_k follows Gaussian distribution with zero mean and covariance matrix.

The resultant average mean squared error and effective sample size are shown in the following table:

Table 1: Algorithm Performance under Different Noise Levels

Algorithm	Particle num.	$\sigma_z = 2$		$\sigma_z = 1$		$\sigma_z = 0.5$	
		Avg. MSE	Avg. ESS	Avg. MSE	Avg. ESS	Avg. MSE	Avg. ESS
PF-PF (LEDH)	200	0.6113	27.98	0.2507	23.25	0.1049	17.40
PF-PF (EDH)	200	0.6024	28.42	0.2539	23.08	0.1060	17.15
PF-PF (EDH)	10^4	0.5224	1035.44	0.2162	904.76	0.0924	698.08
EDH	200	0.4917	N/A	0.1847	N/A	0.0717	N/A
KF	N/A	0.4924	N/A	0.1843	N/A	0.0714	N/A
UKF	N/A	0.4924	N/A	0.1843	N/A	0.0714	N/A
BPF	200	1.4649	1.89	1.3894	1.18	1.3613	1.04
BPF	10^5	0.5672	38.72	0.3929	3.05	0.3632	1.34

Here the average mean squared error is calculated be $\text{Avg.MSE} = \frac{1}{BK} \sum_{b=1}^B \sum_{k=1}^K \|z_{bk} - \hat{z}_{bk}\|^2$, where b is the batch index and k is the time index.

To reveal the advantages of the invertible particle flow particle filter, we test these algorithms on the following non-linear state-space models. Considering tracking the trajectories of four moving targets in a region with 25 acoustic sensors measuring the positions of the targets. A sample is shown in Fig. (a), and the comparison between algorithms are shown in Fig. (b), where it is clear that the local EDH invertible flow outperforms the classic LEDH flow filter and other filters.

3.4 Kernel Particle Flow Filter

We define the flow as $\frac{d\vec{x}_s}{ds} = \vec{f}_s(\vec{x}_s)$ for $s \in [0, 1]$. The boundary conditions are given by:

$$\begin{cases} q_0(\vec{x}) = p(\vec{x}) \\ q_\infty(\vec{x}) = p(\vec{x}|\vec{y}) \end{cases} \quad (72)$$

The flow field $\vec{f}_s$ minimizes the distance between the current distribution and the target. To measure this distance, we use the Kullback-Leibler (KL) divergence:

$$KL(q_s) = \int q_s(\vec{x}) \log \frac{q_s(\vec{x})}{q_\infty(\vec{x})} d\vec{x} \quad (73)$$

We should let $\frac{dKL(q_s)}{ds} \leq 0$. Evaluating this derivative:

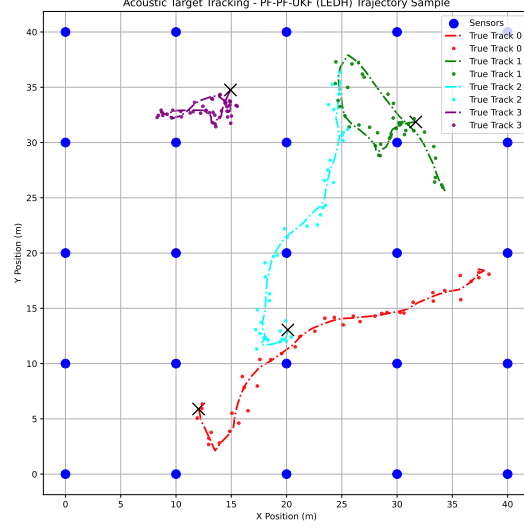
$$\frac{dKL}{ds} = \int \frac{\partial q_s(\vec{x})}{\partial s} \left\{ \log \frac{q_s(\vec{x})}{p(\vec{x}|\vec{y})} + 1 \right\} d\vec{x} \leq 0 \quad (74)$$

Assuming only an advection term, where $d\vec{x}_s = \vec{f}_s(\vec{x}_s)ds$, the density follows the Liouville equation:

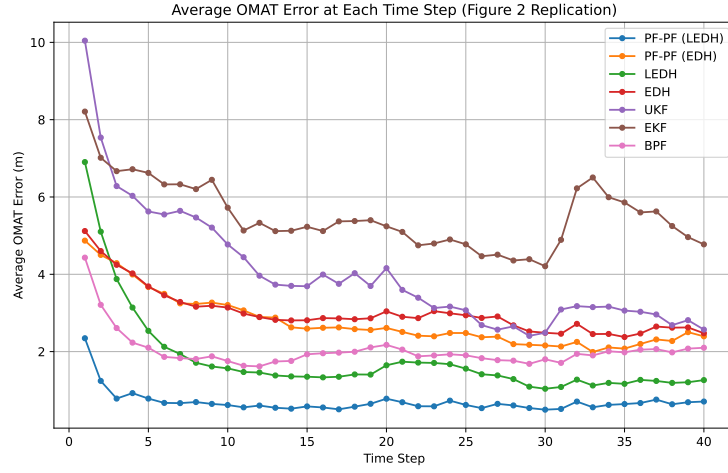
$$\frac{\partial q_s(\vec{x})}{\partial s} + \nabla_{\vec{x}} \cdot (\vec{f}_s(\vec{x})q_s(\vec{x})) = 0 \quad (75)$$

Substituting this back into the derivative:

$$\frac{dKL}{ds} = - \int \nabla_{\vec{x}} \cdot (\vec{f}_s(\vec{x})q_s(\vec{x})) \left\{ \log \frac{q_s(\vec{x})}{p(\vec{x}|\vec{y})} + 1 \right\} d\vec{x} \quad (76)$$



(a)



(b)

Figure 7: (a) A sample of the real and estimated trajectories (by PF-PF (LEDH)). (b) The OMAT for all the algorithms at different time steps averaged over batches.

Assuming $q_s(\vec{x}) \rightarrow 0$ as $\vec{x} \rightarrow \infty$, we integrate by parts:

$$\frac{dKL}{ds} = \int \vec{f}_s(\vec{x}) q_s(\vec{x}) \cdot \nabla_{\vec{x}} \left\{ \log \frac{q_s(\vec{x})}{p(\vec{x}|\vec{y})} + 1 \right\} d\vec{x} \quad (77)$$

$$= \int \vec{f}_s(\vec{x}) q_s(\vec{x}) \cdot \nabla_{\vec{x}} \{ \log q_s(\vec{x}) - \log p(\vec{x}|\vec{y}) \} d\vec{x} \quad (78)$$

$$= \int \left[\vec{f}_s(\vec{x}) \cdot \nabla_{\vec{x}} q_s(\vec{x}) - \vec{f}_s(\vec{x}) q_s(\vec{x}) \cdot \nabla_{\vec{x}} \log p(\vec{x}|\vec{y}) \right] d\vec{x} \quad (79)$$

$$= - \int q_s(\vec{x}) \left[\nabla_{\vec{x}} \cdot \vec{f}_s(\vec{x}) + \vec{f}_s(\vec{x}) \cdot \nabla_{\vec{x}} \log p(\vec{x}|\vec{y}) \right] d\vec{x} \quad (80)$$

Now assume $\vec{f}_s(\vec{x})$ is embedded in a reproducing kernel Hilbert space, such that $\vec{f}_s(\vec{x}) = \langle K(\vec{x}, \cdot), \vec{f}_s(\cdot) \rangle$. Using this property:

$$\nabla_{\vec{x}} \cdot \vec{f}_s(\vec{x}) = \nabla_{\vec{x}} \cdot \langle K(\vec{x}, \cdot), \vec{f}_s(\cdot) \rangle = \langle \nabla_{\vec{x}} \cdot K(\vec{x}, \cdot), \vec{f}_s(\cdot) \rangle \quad (81)$$

Substituting this into the integral yields:

$$\frac{dKL}{ds} = \left\langle - \int q_s(\vec{x}) \{ \nabla_{\vec{x}} \cdot K(\vec{x}, \cdot) + K(\vec{x}, \cdot) \nabla_{\vec{x}} \log p(\vec{x}|\vec{y}) \} d\vec{x}, \vec{f}_s(\cdot) \right\rangle \quad (82)$$

To ensure $\frac{dKL}{ds} \leq 0$, we choose $\vec{f}_s(\cdot) = DI_f$, where:

$$I_f = \int q_s(\vec{x}) \{ \nabla_{\vec{x}} \cdot K(\vec{x}, \cdot) + K(\vec{x}, \cdot) \nabla_{\vec{x}} \log p(\vec{x}|\vec{y}) \} d\vec{x} \quad (83)$$

Thus, $\frac{dKL}{ds} = -\langle I_f, DI_f \rangle \leq 0$ as long as D is positive definite.

Using the empirical particle representation $q_s(\vec{x}) = \frac{1}{N_p} \sum_{i=1}^{N_p} \delta(\vec{x} - \vec{x}_s^i)$:

$$\vec{f}_s(\cdot) = \frac{1}{N_p} D \sum_{i=1}^{N_p} \{ \nabla_{\vec{x}_s^i} \cdot K(\vec{x}_s^i, \cdot) + K(\vec{x}_s^i, \cdot) \nabla_{\vec{x}_s^i} \log p(\vec{x}_s^i|\vec{y}) \} \quad (84)$$

The discrete update for particle j is:

$$\vec{x}_{s+\Delta s}^j = \vec{x}_s^j + \frac{\Delta s}{N_p} D \sum_{i=1}^{N_p} \{ K(\vec{x}_s^i, \vec{x}_s^j) \nabla_{\vec{x}_s^i} \log p(\vec{x}_s^i|\vec{y}) + \nabla_{\vec{x}_s^i} K(\vec{x}_s^i, \vec{x}_s^j) \} \quad (85)$$

Here, the first term is the attracting term, which drives states toward the local maximum of $p(\vec{x}|\vec{y})$, weighted by $K(\vec{x}_s^i, \vec{x}_s^j)$. The second term is the repelling term, which separates particles from each other to prevent making all particles collapse into the mode.

The smoothed gradient at $\vec{x}$ is:

$$\nabla_{\vec{x}} \log p(\vec{x}|\vec{y}) = \nabla_{\vec{x}} \log p(\vec{y}|\vec{x}) + \nabla_{\vec{x}} \log p(\vec{x}) \quad (86)$$

Assume a Gaussian observation error: $\log p(\vec{y}|\vec{x}) \propto -\frac{1}{2} \|\vec{y} - H(\vec{x})\|_R^2$, then:

$$\nabla_{\vec{x}} \log p(\vec{y}|\vec{x}) = \left(\frac{dH}{d\vec{x}} \right)^T R^{-1} (\vec{y} - H(\vec{x})) \quad (87)$$

Since it is $\nabla_{\vec{x}_s^i} \log p(\vec{x}_s^i|\vec{y})$ in $\vec{f}$, $\frac{dH}{d\vec{x}}$ can be evaluated locally for each particle. Any form of $p(\vec{y}|\vec{x})$ and $p(\vec{x})$ can be used in PFF as long as $\nabla_{\vec{x}} \log p$ can be easily obtained.

The Choice of Kernel

For a scalar kernel: $K(\vec{x}, \vec{z}) = K(\vec{x}, \vec{z}) I_{\vec{x}}$. An example is $K(\vec{x}, \vec{z}) = \exp(-\frac{1}{2}(\vec{x} - \vec{z})^T A(\vec{x} - \vec{z}))$, which defines distance between particles in space.

$$\nabla_{\vec{x}} \cdot K(\vec{x}, \vec{z}) = -A^T(\vec{x} - \vec{z}) K(\vec{x}, \vec{z}) \quad (88)$$

The partial derivative evaluates to terms involving $\exp(-\frac{1}{2}w_i A_{ij} w_j)$. The value of the kernel is the same for all components of the two given particles. It can be viewed as a generalized distance between particles $\rightarrow$ a single distance in the whole space. Because $(\vec{x} - \vec{z}) \cdot \nabla_{\vec{x}} K(\vec{x}, \vec{z}) \leq 0$, the projection of $\nabla_{\vec{x}} K(\vec{x}, \vec{z})$ is along the opposite of $\vec{x} - \vec{z}$, and $\nabla_{\vec{x}} \cdot K(\vec{x}, \vec{z})$ acts on $\vec{z}$ along the vector, so it is effectively repelling.

For a matrix kernel: $K(\vec{x}, \vec{z}) = \text{diag}([K_{(1)}(\vec{x}, \vec{z}), K_{(2)}(\vec{x}, \vec{z}), \dots, K_{(n_x)}(\vec{x}, \vec{z})])$, where:

$$K_{(a)}(\vec{x}, \vec{z}) = K_{(a)}(x_{(a)}, z_{(a)}) = \exp\left(-\frac{1}{2} \frac{(x_{(a)} - z_{(a)})^2}{\alpha_a^2}\right) \quad (89)$$

Here, α_a is a tunable factor for the kernel width. The gradient divergence takes the form of independent component evaluations.

If the convergence rates between components of the state are very different, which is the case when the components are partially observed:

- Observed components: Faster convergence.

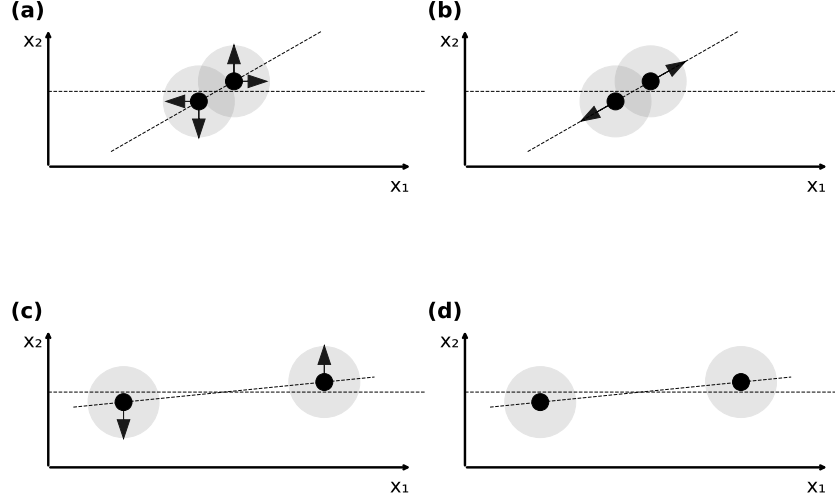


Figure 8: Comparison between scalar and matrix kernel

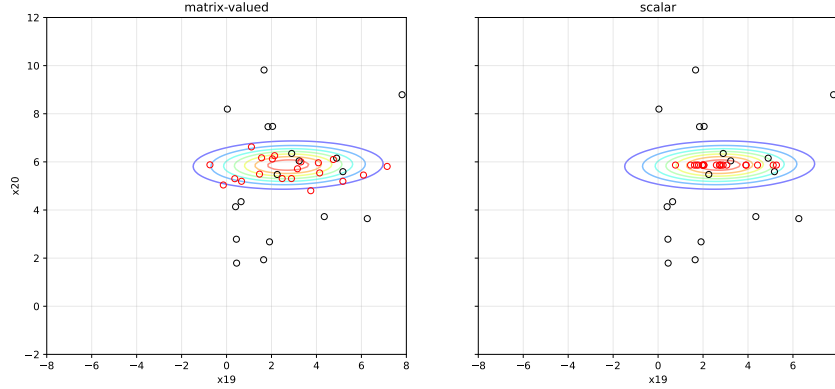


Figure 9: Comparison between the posterior given by matrix kernel and scalar kernel

- Unobserved components: Slower convergence.

Scalar kernel: The kernel value can still be small due to the unobserved components, even if the observed components are close. The repelling force is too weak to separate particles away in the observed component, leading to particles collapsing toward the mode of the posterior.

Matrix kernel: The observed components feel each other independently of the unobserved components. Thus, for the observed components close to each other, the kernel value can be large and the repelling force can be effective even if the unobserved components are far away, avoiding collapse on the posterior mode.

The choice of kernel $K(x, x')$ is critical. A scalar-valued kernel, $K(x, x') = k(x, x')\mathbf{I}$, imposes an isotropic geometry, which is often inefficient for high-dimensional states where the probability mass concentrates on thin manifolds. A matrix-valued kernel $K(x, x') = [K_1(x_1, x'_1), \dots, K_D(x_D, x'_D)]$ incorporates geometric information, typically using the covariance matrix $\mathbf{B}$ as a preconditioner and enable spatial scales to be different along different directions. This allows the flow to account for correlations and varying scales of state variables, significantly improving efficiency in high-dimensional applications, where scalar kernel can give abnormally small repulsive force even if the particles are already close in some dimensions [Fig. 8], and thus lead to particle collapsing problem, as shown in Fig. 9.

4 Learning Model Parameters from Data

4.1 Classic Filters

4.1.1 Rauch-Tung-Striebel (RTS) Smoother

The RTS smoother computes the full posterior marginals $p(x_t|y_{1:T})$ utilizing the entire observation sequence. It operates as a backward pass starting from $t = T - 1$ down to 0. Using the Markov property of the SSM, the smoothing gain J_t is derived as $J_t = P_{t|t}A^\top P_{t+1|t}^{-1}$. The smoothed mean and covariance are then propagated backward according to

$$\hat{x}_{t|T} = \hat{x}_{t|t} + J_t(\hat{x}_{t+1|T} - \hat{x}_{t+1|t}) \quad (90)$$

$$P_{t|T} = P_{t|t} + J_t(P_{t+1|T} - P_{t+1|t})J_t^\top \quad (91)$$

Additionally, the cross-covariance $P_{t,t-1|T}$, which is required for parameter learning via EM, is approximated by $P_{t,t-1|T} \approx P_{t|T}J_{t-1}^\top$.

4.1.2 EM Algorithm for LGSSM

To estimate the model parameters $\theta = \{A, C, Q, R, \mu_0, P_0\}$, we maximize the log-likelihood $\mathcal{L}(\theta) = \log p(y_{1:T}|\theta)$. The Expectation-Maximization (EM) algorithm maximizes the expected complete log-likelihood $\mathcal{Q}(\theta, \theta^{old}) = \mathbb{E}_{X|Y}[\log p(x_{0:T}, y_{1:T}|\theta)]$. By taking derivatives of $\mathcal{Q}$ with respect to the parameters and setting them to zero, we obtain analytic solutions implemented in `EM_solver`. Letting the sufficient statistics be $\Sigma_{xx} = \sum_{t=0}^{T-1} \mathbb{E}[x_t x_t^\top]$, $\Sigma_{x1x1} = \sum_{t=1}^T \mathbb{E}[x_t x_t^\top]$, $\Sigma_{xx1} = \sum_{t=1}^T \mathbb{E}[x_t x_{t-1}^\top]$, and $\Gamma_{yx} = \sum_{t=0}^{T-1} y_t \mathbb{E}[x_t]^\top$, the update rules for the M-step are

$$A_{new} = \Sigma_{xx1} \Sigma_{xx}^{-1} \quad (92)$$

$$C_{new} = \Gamma_{yx} \Sigma_{xx}^{-1} \quad (93)$$

$$Q_{new} = \frac{1}{T-1} (\Sigma_{x1x1} - A_{new} \Sigma_{xx1}^\top - \Sigma_{xx1} A_{new}^\top + A_{new} \Sigma_{xx} A_{new}^\top) \quad (94)$$

$$R_{new} = \frac{1}{T} \left(\sum y_t y_t^\top - C_{new} \Gamma_{yx}^\top - \Gamma_{yx} C_{new}^\top + C_{new} \Sigma_{xx} C_{new}^\top \right) \quad (95)$$

Fig. 10 shows an example of the convergence of the EM algorithm, where the model is a linear Gaussian state-space model with three state and observation dimensions, and the model parameters are drawn from independent normal distributions

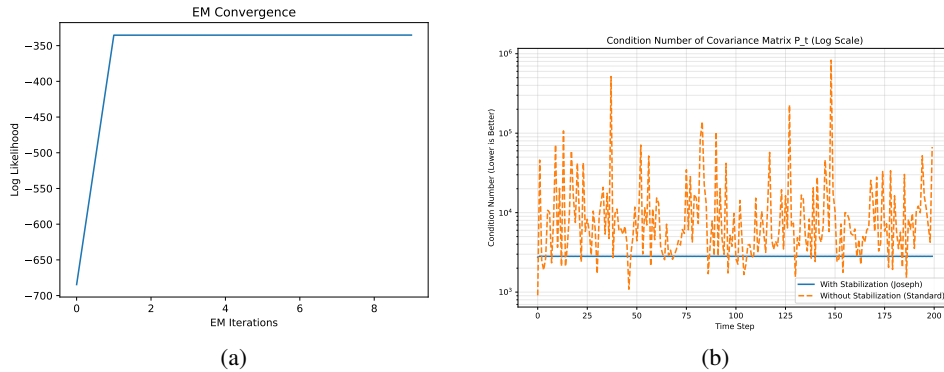


Figure 10: (a) EM algorithm convergence example for Kalman filter. (b) Effect of the Joseph stabilization for the posterior covariance update

4.2 Differentiable Particle Filter

Particle Filtering (PF) methods are powerful procedures for performing inference in non-linear state-space models. A key ingredient of PF is the resampling step, which is necessary to replicate

particles with high weights and discard those with low weights, thereby ensuring low variance in the likelihood and state estimates.

However, traditional resampling schemes (such as multinomial, stratified, or systematic resampling) introduce a fundamental limitation: they result in PF-based loss functions being non-differentiable with respect to the model parameters θ and the proposal parameters ϕ . In a variational inference context, standard resampling relies on discrete sampling, which prevents the flow of gradients and yields high-variance gradient estimates for the PF-based evidence lower bound (ELBO). This effectively hinders end-to-end training via backpropagation.

To bypass the non-differentiability of traditional resampling, Optimal Transport (OT) is introduced. Standard resampling only considers particle weights, but OT considers both the weights and the physical locations of the particles in the state space.

By finding an Optimal Transport plan between the empirical distribution of the proposed particles α_N and the weighted target posterior β_N , a deterministic map, the Ensemble Transform (ET), can be defined to transport the particles to their new resampled locations. While the standard Ensemble Transform solves this via a linear program, the exact OT cost is computationally expensive ($\mathcal{O}(N^3 \log N)$) and the resulting map remains non-differentiable.

4.2.1 Entropy-Regularized Optimal Transport

To achieve both computational efficiency and differentiability, the Differentiable Particle Filter (DPF) introduces an entropy-regularized version of the OT problem.

Let $\alpha_N = \sum_{i=1}^N a_i \delta_{u_i}$ and $\beta_N = \sum_{j=1}^N b_j \delta_{v_j}$ be the source and target atomic distributions with weight vectors a and b . The squared 2-Wasserstein metric with entropy regularization $\epsilon > 0$ is formulated as:

$$\mathcal{W}_{2,\epsilon}^2(\alpha_N, \beta_N) = \min_{P \in \mathcal{S}(a,b)} \sum_{i,j=1}^N p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \quad (96)$$

where $c_{i,j} = \|u_i - v_j\|^2$ is the cost matrix and $\mathcal{S}(a,b) = \{P \in [0,1]^{N \times N} : P\mathbf{1} = a, P^T \mathbf{1} = b\}$ is the constraint of the transport matrix P_ϵ^{OT} imposed by the two distributions. The first term is the cost function of the distribution transformation. The second entropy term ensures a unique optimal transport matrix P_ϵ^{OT} .

4.2.2 Sinkhorn Algorithm

To solve this efficiently, we derive the dual formulation. We construct the Lagrangian $\mathcal{L}$ by introducing dual multipliers $f \in \mathbb{R}^N$ and $g \in \mathbb{R}^N$ for the marginal constraints $\sum_j p_{i,j} = a_i$ and $\sum_i p_{i,j} = b_j$:

$$\begin{aligned} \mathcal{L}(P, f, g) = & \sum_{i,j} p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \\ & - \sum_i f_i \left(\sum_j p_{i,j} - a_i \right) - \sum_j g_j \left(\sum_i p_{i,j} - b_j \right) \end{aligned} \quad (97)$$

Taking the derivative with respect to the primal variable $p_{i,j}$ and setting it to zero yields:

$$\frac{\partial \mathcal{L}}{\partial p_{i,j}} = c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} + \epsilon - f_i - g_j = 0 \quad (98)$$

Solving for $p_{i,j}$, and absorbing the constant ϵ shift into the dual variables, we obtain the regularized transport matrix:

$$p_{\epsilon,i,j}^{OT} = a_i b_j \exp \left(\frac{f_i^* + g_j^* - c_{i,j}}{\epsilon} \right) \quad (99)$$

Substituting this optimal $p_{i,j}$ back into the Lagrangian provides the unconstrained dual optimization problem:

$$\max_{f,g} DOT_\epsilon(f,g) := a^T f + b^T g - \epsilon a^T M b \quad (100)$$

where $M_{i,j} = \exp(\epsilon^{-1}(f_i + g_j - c_{i,j})) - 1$.

The dual objective is concave and unconstrained. Setting the gradients with respect to f and g to zero provides the first-order optimality conditions:

$$\nabla_f DOT_\epsilon = 0 \implies f_i^* = \mathcal{T}_\epsilon(b, g^*, C_{i:}) \quad (101)$$

$$\nabla_g DOT_\epsilon = 0 \implies g_i^* = \mathcal{T}_\epsilon(a, f^*, C_{:i}) \quad (102)$$

where $\mathcal{T}_\epsilon$ is the soft-min operator mapping:

$$\mathcal{T}_\epsilon(a, f, C_{:,i}) = -\epsilon \log \sum_k \exp \{ \log a_k + \epsilon^{-1} (f_k - c_{k,i}) \} \quad (103)$$

The Sinkhorn algorithm solves the OT problem by alternating coordinate ascent on these dual variables f and g until convergence.

4.2.3 Differentiability

Once P_ϵ^{OT} is found, the resampled particles are obtained via the Differentiable Ensemble Transform (DET):

$$\tilde{X}_t^i = N \sum_{k=1}^N p_{\epsilon,i,k}^{OT} X_t^k := T_{N,\epsilon}^{(t)}(X_t^i) \quad (104)$$

Because the Sinkhorn iterations only involve basic, differentiable arithmetic operations (such as log-sum-exp), the final dual vectors (f^*, g^*) can be differentiated natively using automatic differentiation through the algorithm loop. Alternatively, they can be differentiated more efficiently using the implicit function theorem (gradient stitching) on the optimality conditions at convergence. The exact derivatives of P_ϵ^{OT} are then explicitly available by differentiating the explicit exponential form, avoiding discrete sampling steps entirely.

The resulting DET accurately approximates the target posterior. Theoretical bounds show that for any bounded 1-Lipschitz function ψ , the difference between the empirical target β_N and the OT-resampled distribution $\tilde{\beta}_N$ is bounded by a function of the Wasserstein distances and the entropic penalty.

Specifically, as the number of particles $N \rightarrow \infty$ and if the regularization parameter ϵ_N is chosen appropriately, the error bound vanishes entirely. Furthermore, consistency results for the DPF prove that the resulting particle approximations $\tilde{\beta}_N^{(t)}$ converge in probability to the true filtering distribution $\beta^{(t)} = p(x_t | y_{1:t})$. Thus, the differentiable transport dynamically shifts the particles to successfully mimic exact sampling from the correct posterior density.

Table 2: Mean and standard deviation of $\frac{1}{T}(\hat{l}(\theta; U) - l(\theta))$

Method	$\theta_1, \theta_2 = 0.25$	0.5	0.75
PF mean	-0.54	-0.47	-0.51
PF std	0.10	0.11	0.10
DPF ($\epsilon = 0.25$) mean	-0.54	-0.48	-0.54
DPF ($\epsilon = 0.25$) std	0.12	0.12	0.10
DPF ($\epsilon = 0.5$) mean	-0.54	-0.47	-0.54
DPF ($\epsilon = 0.5$) std	0.12	0.12	0.11
DPF ($\epsilon = 0.75$) mean	-0.54	-0.47	-0.55
DPF ($\epsilon = 0.75$) std	0.10	0.10	0.11

To

The function `create_stable_nlsfm()` defines a one-dimensional non-linear state-space model with the following components:

Notes

- The sine-based transition ensures bounded dynamics, preventing divergence.

Table 3: Training Progress

Step	Loss	ϕ RMSE	Avg ESS (%)
001	258.85	0.9700	11.5
010	210.97	0.6672	16.5
020	189.73	0.3831	29.6
030	186.08	0.1922	52.0
040	184.48	0.1249	63.8
050	185.23	0.1296	65.9

Component	Definition	Explanation
State dimension (d_x)	1	The latent state is scalar.
Observation dimension (d_y)	1	Observations are scalar.
Transition function	$x_t = 0.9 \cdot \sin(x_{t-1}) + \eta_t$	Non-linear transition with sine ensures stability around $x = 0$. η_t is process noise.
Observation function	$y_t = \frac{1}{10}x_t + \epsilon_t$	Observations are proportional to the state with added noise.
Process noise	$\eta_t \sim \mathcal{N}(0, \sqrt{2})$	Multivariate normal with diagonal covariance, variance = 2.
Observation noise	$\epsilon_t \sim \mathcal{N}(0, \sqrt{2})$	Measurement noise with variance = 2.
Initial noise	$x_0 \sim \mathcal{N}(0, \sqrt{0.1})$	Prior uncertainty on the initial state.
Initial state	$x_0 = 0$	The deterministic starting point before noise.

Table 4: Summary of the NLSSM model and parameters.

- The scaling factor 0.9 dampens oscillations, reinforcing stability.
- Noise terms are modeled as Gaussian with specified variances, reflecting uncertainty in both dynamics and observations.
- The model is suitable for testing non-linear filtering methods such as the Extended Kalman Filter or Particle Filters.

Epsilon	Bias (State Error)	Variance (State Fluctuation)	Metric
0.01	2.1259	0.0475	49.33
0.10	2.1366	0.0489	49.33
0.50	2.1198	0.0506	49.33
1.00	2.1068	0.0543	49.33
5.00	2.0183	0.0838	49.39

Table 5: Bias-variance tradeoff across different values of ϵ . The fourth column represents an additional performance metric.